

Multi Agent Reinforcement Learning For Flatland Environment

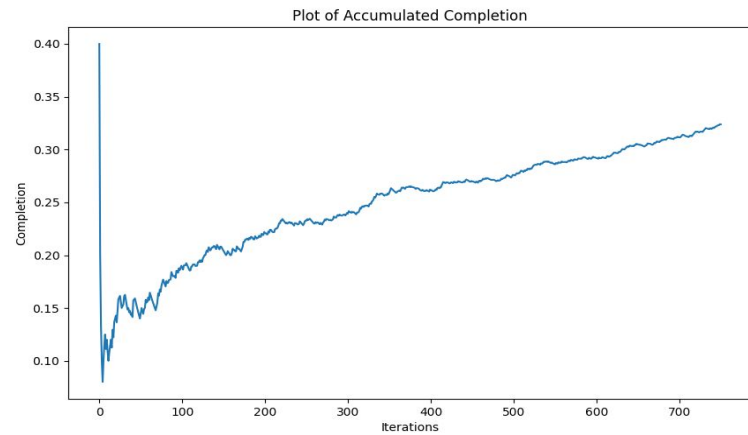
K.S MOHAN KUMAR(ME19B118)



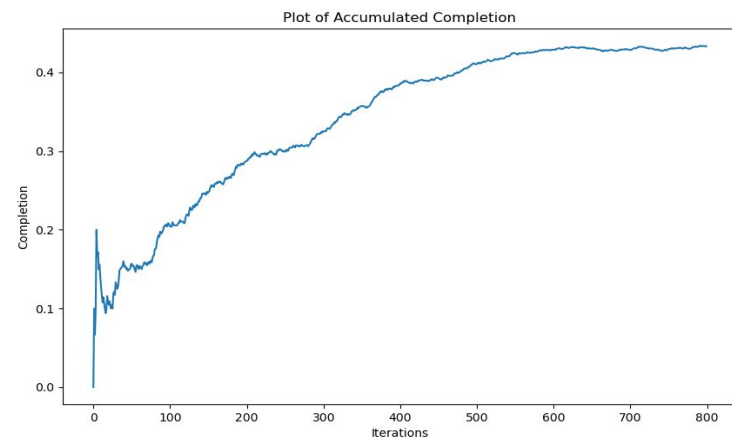
Flatland Environment

- ❑ The time in Flatland3 is discretized into timestamps from 0 to $T_{\max}$. There are N trains and multiple cities.
- ❑ Trains are numbered from 1 to N . Trains' action space includes five actions: do nothing, move forward, stop, turn left, turn right. Backward movement is not allowed, and trains must follow the rails.
- ❑ Each train i has a unique speed s_i , allowing it to move one step every $1/s_i$ turns. The time cost of one step $1/s_i$ is guaranteed to be an integer, with the maximum speed being 1 step per turn. For each train i , there are defined earliest departure time A_i and latest arrival time B_i .
- ❑ Trains may randomly experience breakdowns (malfunctions) while in motion or waiting for departure. After a breakdown, a train must remain stationary for a certain period.
- ❑ The tree observations exploits the graph nature in the railways environment locally. It computes tree nodes for a given depth, and we flatten the tree to be used as observations for the algorithms.

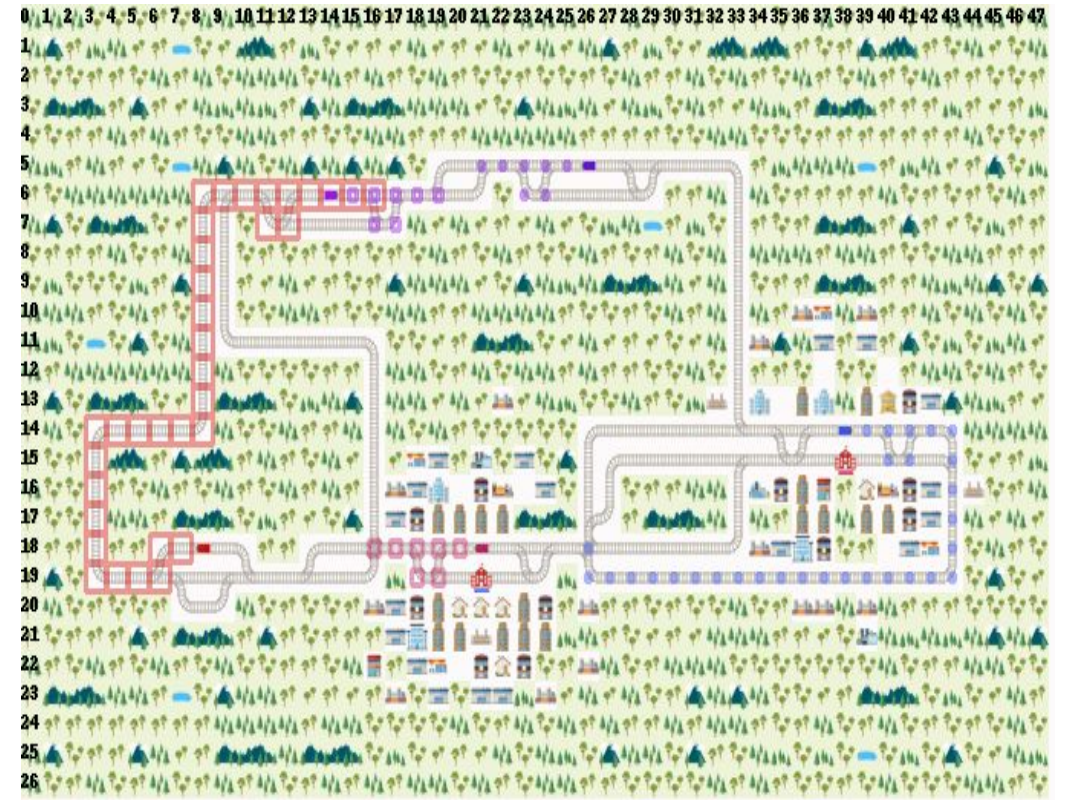
Using D3QN and PS-PPO



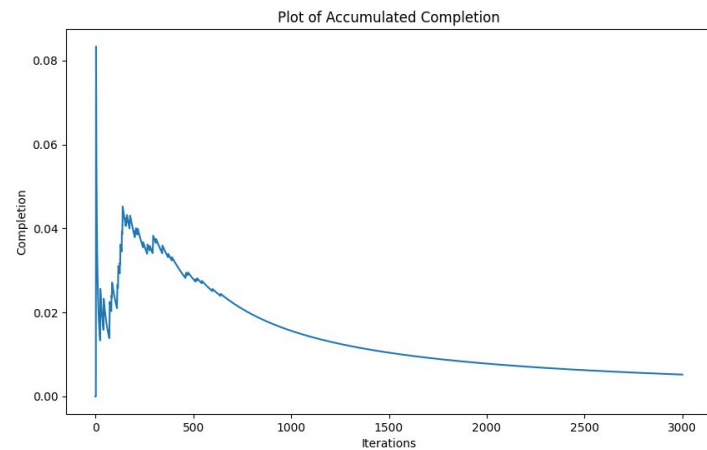
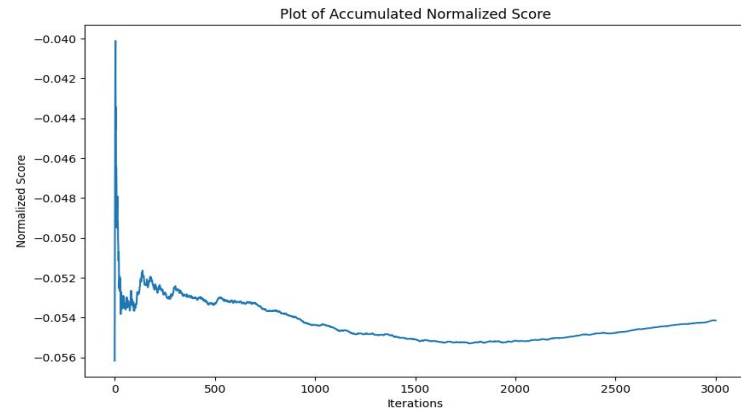
D3QN



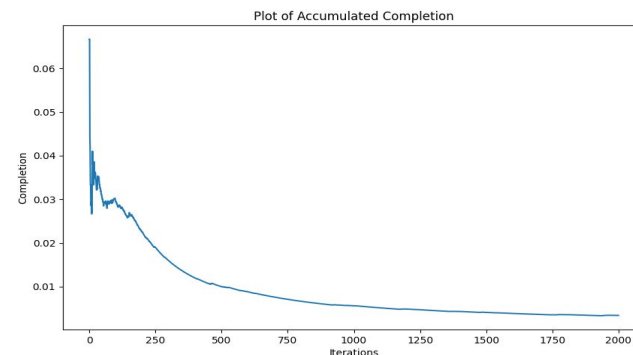
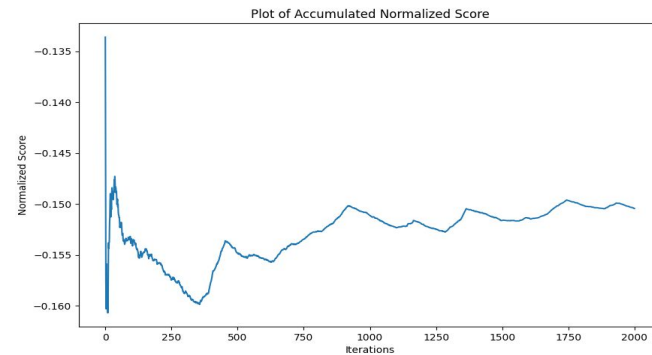
PS-PPO



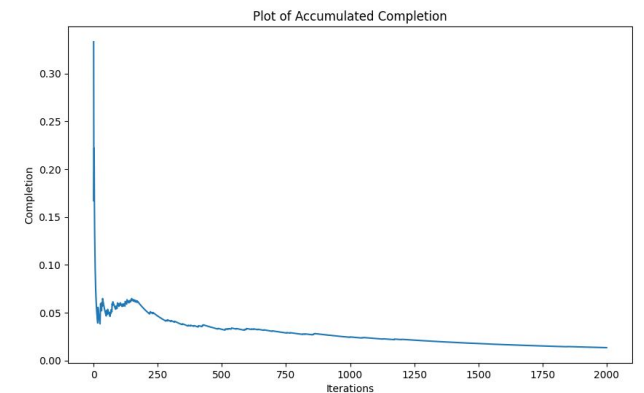
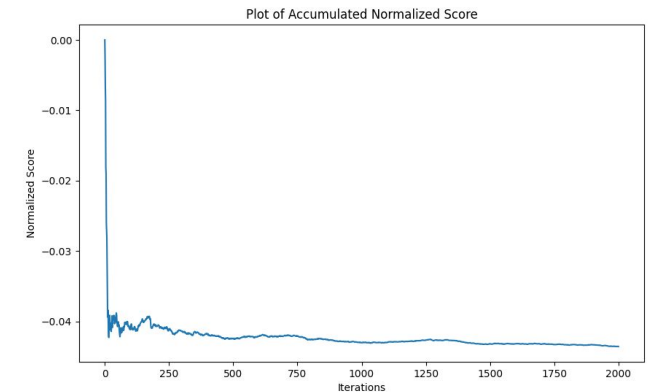
Results with Parameter Sharing PPO



Custom sparse environment with 3 agents and 3 cities

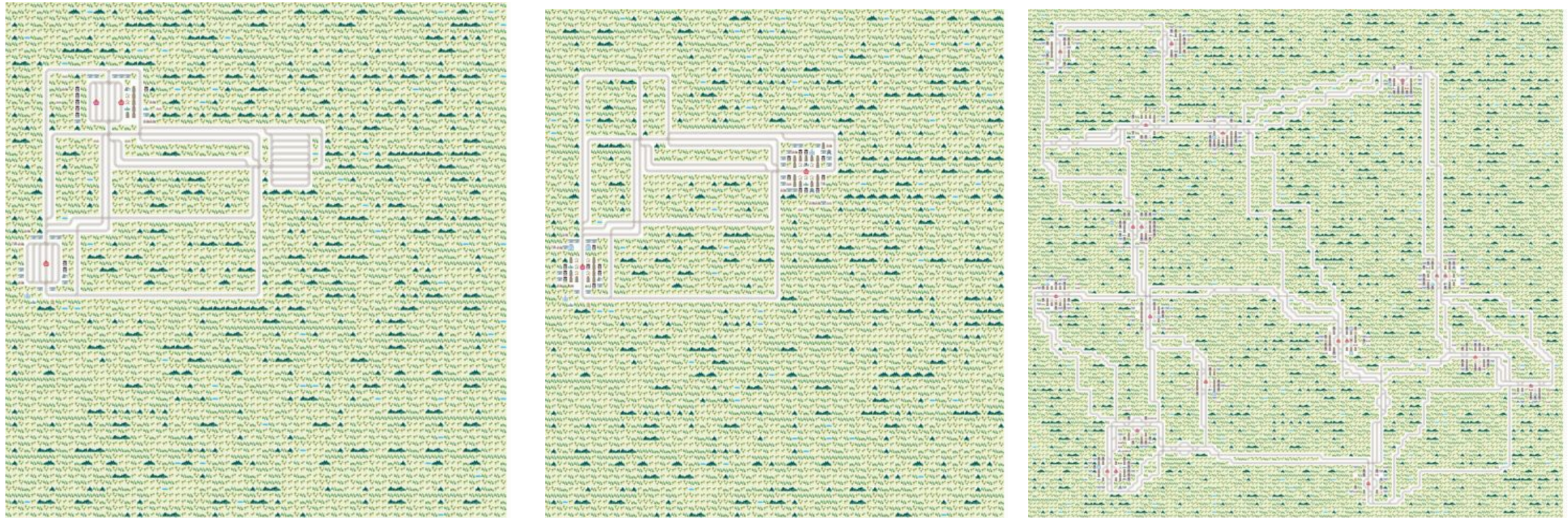


30 agents, 20 cities in 125x125 grid



Custom dense environment with 3 agents and 3 cities

Results from Default Tree Observations using PPO



Tree Observations with LSTMs and Transformers

- ❑ The first part, X^{attr} , are attributes of each agent, such as ID, earliest departure time, latest arrival time, their current state, direction.
- ❑ The second part X^{tree} is the main part of the observations. It encodes possible future paths of each agent as well as useful information about these paths into a tree-like structure.
- ❑ We pass the X^{attr} and X^{tree} into the network architecture and train the agent with standard MA-PPO with CTDE.

Each LSTM cell takes state (c_{t-1}, h_{t-1}) of last cell and a new x_t as input, and output new cell state c and h to next cell.

$$(c_t, h_t) = \text{LSTM-Cell}(x_t, (c_{t-1}, h_{t-1})) \quad (13)$$

$$H^{(0)} = [H^{\text{attr}}, H^{\text{tree}}], \quad (8)$$

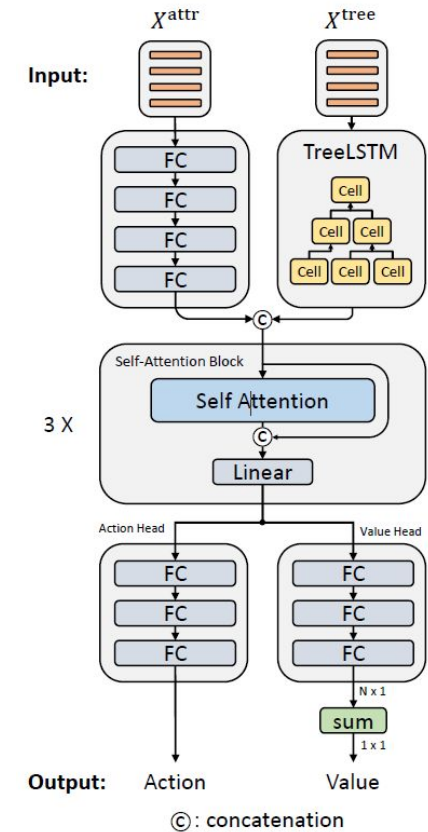
$$H^{(l)} = \text{Self-Attention}(H^{(l-1)}), \quad l = 1, 2, 3. \quad (9)$$

Finally, $H^{(3)}$ is fed into two different heads to obtain final actions logits $A \in \mathbb{R}^{N \times 5}$ and estimated state-value $v \in \mathbb{R}$.

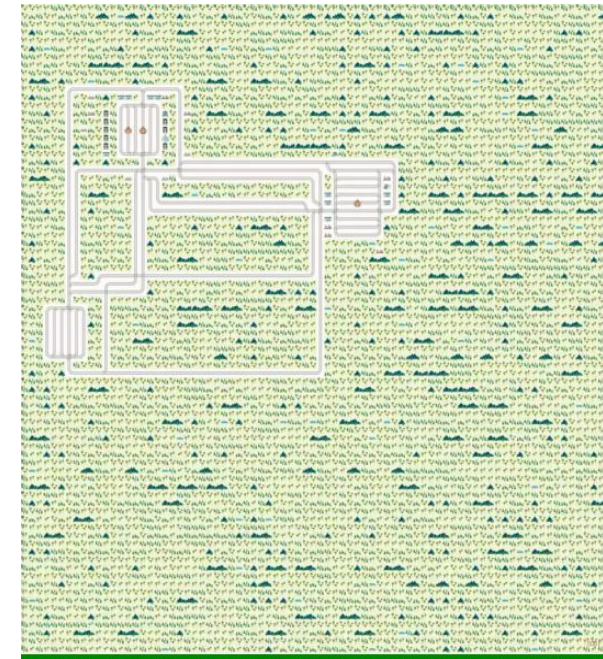
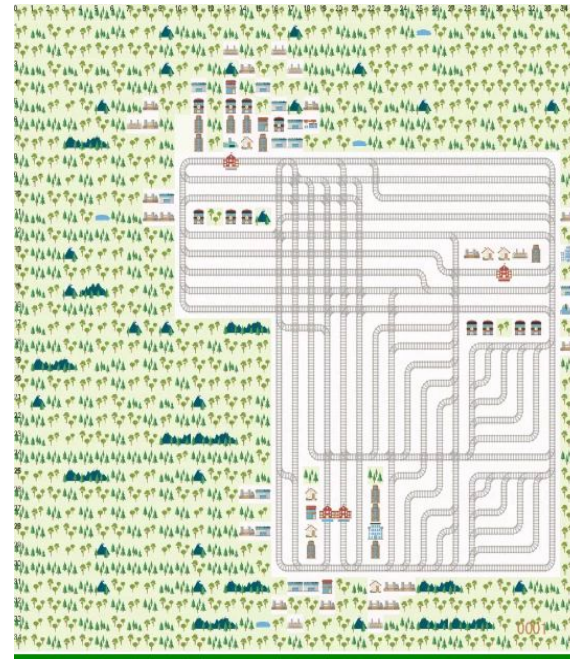
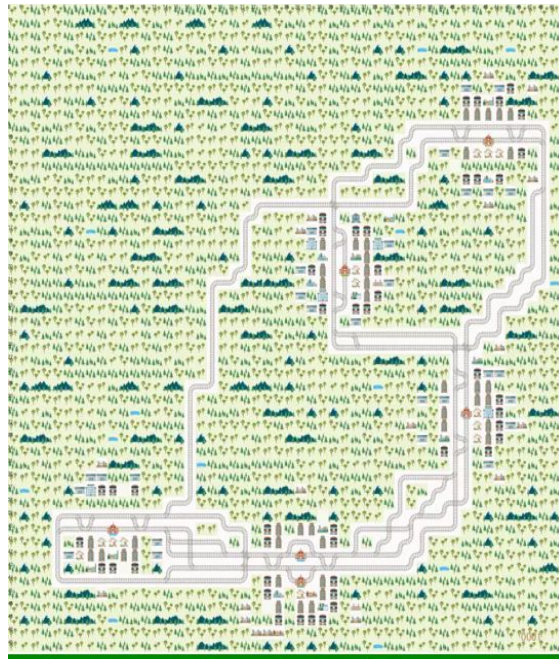
$$A = \text{MLP}(H^{(3)}) \quad (10)$$

$$V = \text{MLP}(H^{(3)}) \quad (11)$$

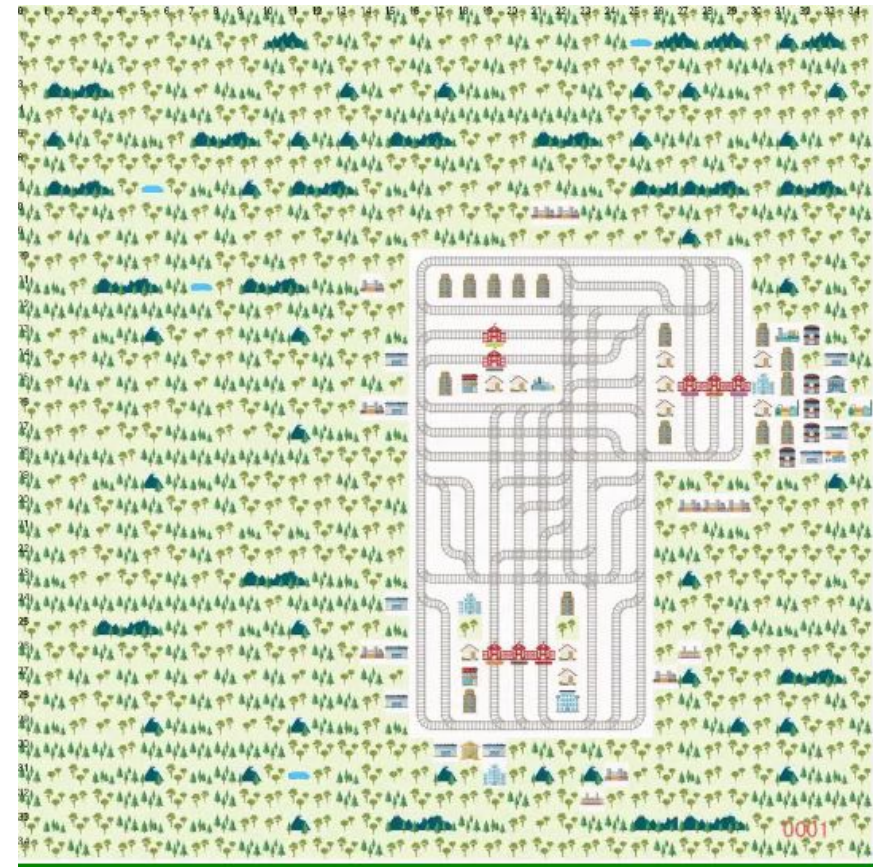
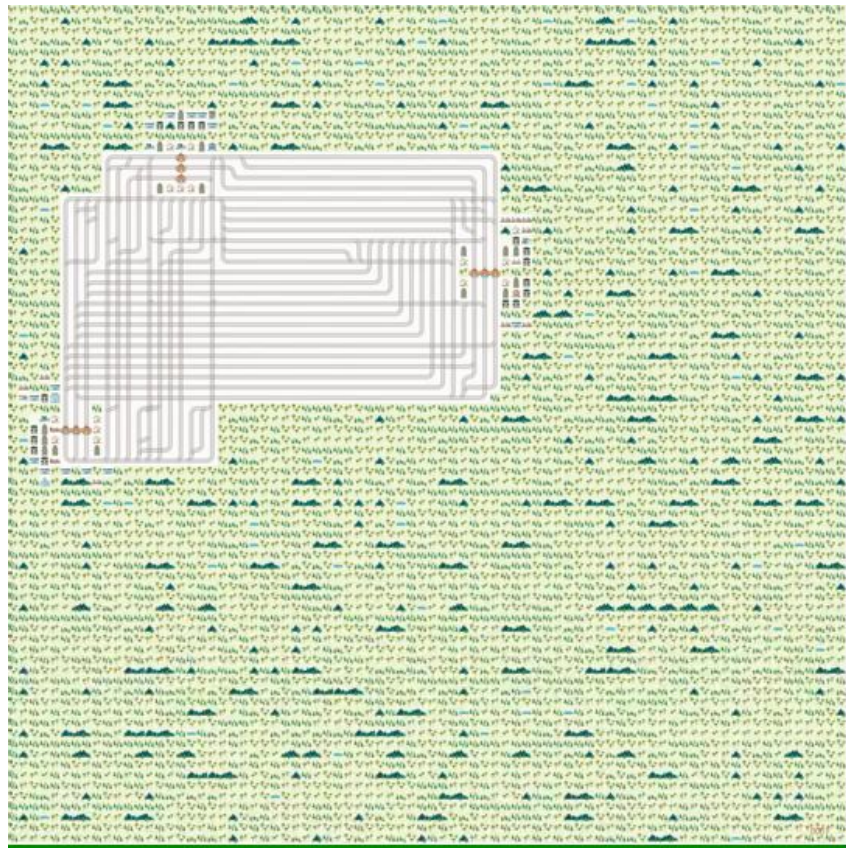
$$v = \sum_{i=0}^N V_i \quad (12)$$



Results from TreeLSTM Observations



Results from Default TreeLSTM Observations



Results and Conclusion

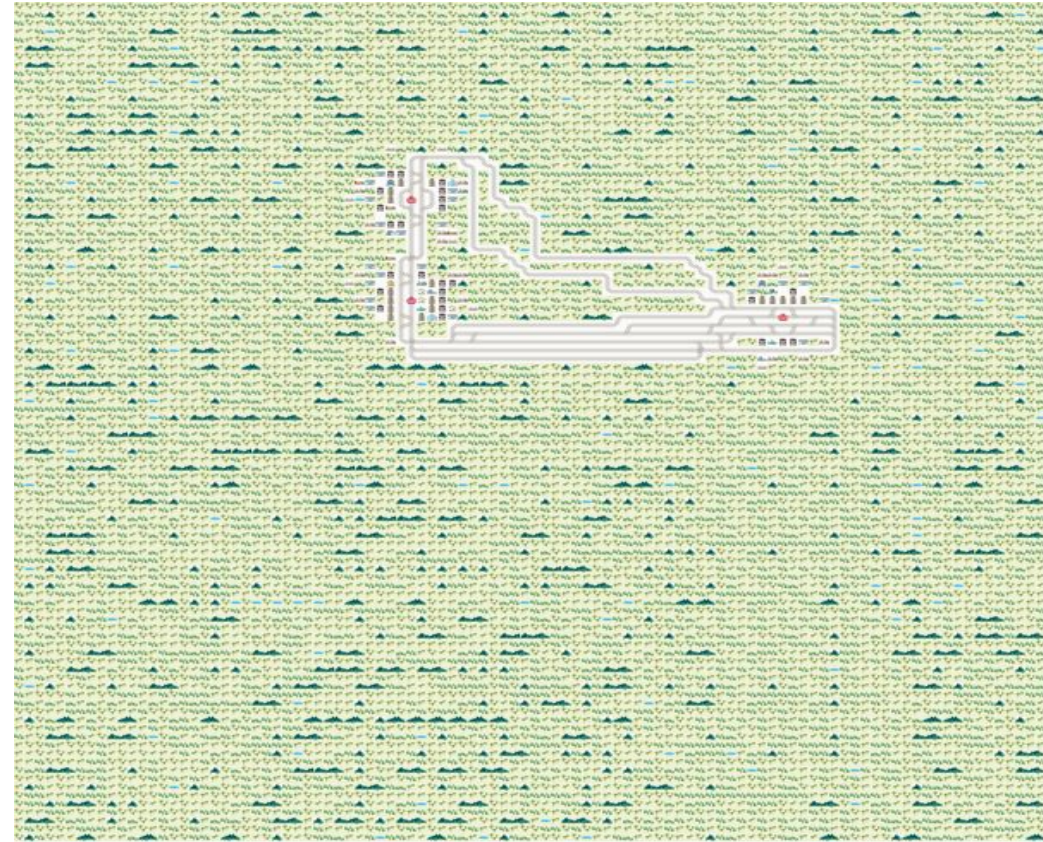
- ❑ It can be seen that from the built-in tree observations performs poorly in spaces where the network is dense. It often gets into loops and do not reach their destinations even after applying reward shaping.
- ❑ But however we can see that when using the TreeLSTM based observation we have significantly great results for the same configuration of the environment with more number of agents.
- ❑ This is because of the Transformer encoder which captures the heatmap of the traffic around it based on a predictor that utilizes the observations collected in the form of spanning tree.
- ❑ It is also understood that simply using the HeatMap Observations does not significantly increase the performance of our agent as the builtin tree observations are capturing the similar observation but in a different fashion.

CONFLICT BASED SEARCH WITH REPLANNING

- ❑ CBS is a two-level algorithm that guarantees optimal paths for each agent in a map internally represented as a graph.
- ❑ **Initialization:** The algorithm starts with an empty constraint tree and a set of agents with their initial positions.
- ❑ **High-Level Search:** The high-level search generates a new node in the CT for each conflict found between agents.
- ❑ **Low-Level Search:** The low-level search is conducted for each agent to find an optimal path with the given constraints.
- ❑ **Conflict Resolution:** The algorithm resolves conflicts by generating constraints for the agents involved in the collision.
- ❑ **Termination:** The algorithm terminates when all agents have found their optimal paths without conflicts.
- ❑ We then use Dynamic RRT to perform Replanning in the middle of an episode when an agent is in a state of malfunction to modify the constraints and new optimal path.

Results from OR approach

- ❑ For simple environments, CBS provides better results than any Reinforcement Learning solution.
- ❑ Combining CBS with Replanning and Large Neighbourhood Search we can mitigate the stochasticity introduced because of speed of trains and malfunction parameters.
- ❑ The method adopted here is a simpler version of the official winning solution of the Flatland Challenge.



Thank You!

A solid orange horizontal bar at the bottom of the slide.